9

调度：比例份额

本章介绍另一种调度程序（scheduler），叫作**比例份额**（proportional-share）调度程序，有时也叫**公平份额**（fair-share）调度程序。比例份额（proportional-share）基于一个简单的概念：调度程序不去优化周转时间（turnaround time）或响应时间（response time），而是设法保证每个作业获得一定比例的 CPU 时间。

比例份额（proportional-share）调度一个很好的早期例子，出自 Waldspurger 和 Weihl 的研究 [WW94]，叫作**彩票调度**（lottery scheduling）；不过这个想法肯定出现得更早 [KL88]。基本思想很简单：每隔一段时间，就抽一次奖，决定下一个该运行哪个进程（process）；应该更常运行的进程，就给它更多中奖的机会。很简单，对吧？下面来看细节！不过先看我们的关键问题：

关键问题：如何按比例分享 CPU
怎样设计调度程序，才能按比例分享 CPU？实现这一点的关键机制是什么？这些机制效果如何？

9.1 基本概念：彩票代表你的份额

彩票调度（lottery scheduling）的背后，是一个非常基本的概念：**彩票**（tickets）。彩票（ticket）用来表示一个进程——或者用户，或者别的什么——应得的某种资源的份额。一个进程持有的彩票（ticket）占总数的百分比，就代表它在这种系统资源上的份额。

来看一个例子。假设有两个进程 A 和 B，A 有 75 张彩票（ticket），B 只有 25 张。那么我们希望 A 得到 75% 的 CPU，B 得到剩下的 25%。

彩票调度（lottery scheduling）通过每隔一段时间（比如每个时间片（time slice））抽一次奖，以概率的方式（而不是确定的方式）实现这一目标。抽奖很简单：调度程序必须知道彩票（ticket）的总数，在我们的例子中是 100 张。然后，调度程序选出一张中奖彩票（ticket），也就是 0 到 99 之间的一个数[1]。假设 A 持有 0 到 74 号彩票（ticket），B 持有 75 到 99 号，那么中奖彩票就直接决定了是 A 运行还是 B 运行。随后，调度程序载入中奖进程的状态并运行它。

[1]计算机科学家总是从 0 开始计数。这在非计算机人士看来太奇怪了，以至于有名人觉得有必要专门写文章解释我们为什么这么做 [D82]。

提示：利用随机性

彩票调度（lottery scheduling）最美妙的地方之一，就是它对**随机性**（randomness）的运用。需要做决定时，采用这种随机化的方法往往既稳健又简单。

和更传统的决策方法相比，随机方法至少有三个优点。第一，随机方法常常能避开一些奇怪的边界情况，而更传统的算法可能难以应付这些情况。例如 LRU 置换策略（replacement policy），后面讲虚拟内存（virtual memory）的一章会更详细地研究它。LRU 通常是一种不错的置换算法，但在某些循环顺序工作负载（workload）下，它会出现最坏情况下的性能。随机方法就没有这样的最坏情况。

第二，随机方法很轻量，只需记录很少的状态来跟踪各个候选项。在传统的公平份额（fair-share）调度算法中，要记录每个进程已经得到了多少 CPU，就得对每个进程单独记账（accounting），而且每运行完一个进程都要更新。用随机的办法，每个进程只需要最少量的状态（比如它持有多少张彩票（ticket））。

最后，随机方法可以非常快。只要生成随机数够快，做决定也就够快，因此在许多需要速度的地方都可以使用随机方法。当然，对速度的要求越高，所用的随机数就越趋向于伪随机。

下面是一个彩票调度程序（lottery scheduler）抽出的中奖彩票（ticket）的示例输出：

```
63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43  0 49 12
```

由此得到的调度结果如下：

```
A     A  A     A  A  A  A  A  A     A     A  A  A  A  A  A
   B        B                    B     B
```

从这个例子可以看出，彩票调度（lottery scheduling）中随机性的运用，使它在满足预期比例上具有概率意义上的正确性，但并不保证一定满足。在上面的例子中，B 在 20 个时间片里只运行了 4 个（20%），而不是期望的 25%。不过，这两个作业竞争的时间越长，它们就越有可能达到期望的比例。

9.2 彩票机制

彩票调度（lottery scheduling）还提供了一些机制，可以用各种不同的、有时很有用的方式来操纵彩票（ticket）。其中一种是 **ticket currency** 的概念。有了 ticket currency，持

提示：用彩票表示份额

在彩票调度（lottery scheduling）以及步长调度（stride scheduling）的设计中，最强大（也最基本）的机制之一就是**彩票**（ticket）。在这些例子里，彩票（ticket）用来表示进程在 CPU 上的份额，但它的用途要广泛得多。例如，在较新的关于 hypervisor 虚拟内存管理的研究中，Waldspurger 展示了如何用彩票（ticket）来表示客户操作系统在内存上的份额 [W02]。所以，如果你需要一种机制来表示所有权的比例，这个概念也许正是……（卖个关子）……彩票（ticket）。

```
// counter: 用来记录是否已经找到中奖者
int counter = 0;

// winner: 调用某个随机数生成器，
//         得到一个 >= 0 且 <= (totaltickets - 1) 的值
int winner = getrandom(0, totaltickets);

// current: 用它来遍历作业列表
node_t *current = head;
while (current) {
    counter = counter + current->tickets;
    if (counter > winner)
        break; // 找到中奖者了
    current = current->next;
}
// 'current' 就是中奖者：调度它……
```

图 9.1：**彩票调度的决策代码**

有一组彩票（ticket）的用户可以用自己喜欢的任意货币单位，把彩票分配给自己的各个作业；系统随后会自动把这种货币换算成正确的全局值。

例如，假设用户 A 和 B 各分到 100 张彩票（ticket）。用户 A 运行两个作业 A1 和 A2，并以 A 的货币各给它们 500 张彩票（ticket），总共 1000 张。用户 B 只运行一个作业，给了它 10 张彩票（ticket），总共 10 张。系统把 A1 和 A2 各自以 A 的货币计的 500 张，换算成全局货币的 50 张；同样，B1 的 10 张彩票（ticket）被换算成 100 张。然后，在全局货币的全部 200 张彩票（ticket）上抽奖，决定运行哪个作业。

```
User A -> 500 (A's currency) to A1 -> 50 (global currency)
       -> 500 (A's currency) to A2 -> 50 (global currency)
User B -> 10 (B's currency) to B1 -> 100 (global currency)
```

另一种有用的机制是 **ticket transfer**。借助 ticket transfer，一个进程可以把自己的彩票（ticket）暂时交给另一个进程。这种能力在客户端/服务器（client/server）的场景中尤其有用：客户端进程向服务器发送消息，请服务器代它做一些工作。为了加快工作，客户端可以把彩票（ticket）交给服务器，从而在服务器处理客户端请求时，尽量提高服务器的性能。处理完以后，服务器再把彩票（ticket）交还给客户端，一切恢复原样。

最后，**ticket inflation** 有时也是一种有用的技术。借助 ticket inflation，进程可以临时提高或降低自己持有的彩票（ticket）数。当然，在进程之间互不信任的竞争场景中，这样做没什么意义：一个贪心（greedy）的进程可以给自己发大量彩票（ticket），从而霸占整台机器。ticket inflation 适用于一组互相信任的进程所在的环境；在这种情况下，如果某个进程知道自己需要更多 CPU 时间，就可以提高自己的彩票（ticket）数，借此向系统表明这种需要，而且完全不必和其他进程通信。

9.3 实现

彩票调度（lottery scheduling）最令人惊叹的一点，大概就是它实现起来非常简单。你只需要一个好的随机数生成器来选出中奖彩票（ticket），一个数据结构（data structure，比如一个列表）来记录系统中的进程，以及彩票的总数。

假设我们把进程放在一个列表里。下面是一个例子，其中有三个进程 A、B 和 C，各自持有一定数量的彩票（ticket）。



要做出调度决定，我们先得从彩票（ticket）总数（400）中选出一个随机数（也就是中奖号码）[2]。假设我们选中的是 300。接下来，只需遍历这个列表，用一个简单的计数器帮我们找出中奖者（图 9.1，第 3 页）。

这段代码遍历进程列表（process list），把每个进程的彩票（ticket）数累加到 `counter` 上，直到它的值超过 `winner`。一旦超过，当前的列表元素就是中奖者。以中奖彩票（ticket）是 300 为例，过程如下。首先，`counter` 加上 A 的彩票（ticket），变成 100；因为 100 小于 300，循环继续。接着，`counter` 加上 B 的彩票（ticket），变成 150，仍然小于 300，于是循环再次继续。最后，`counter` 变成 400（显然大于 300），于是跳出循环，此时 `current` 指向 C（中奖者）。

为了让这个过程尽可能高效，一般最好把列表按彩票（ticket）数从多到少排好序。排序不影响算法的正确性；不过总体来说，它能确保遍历列表的次数最少，尤其是当少数几个进程持有大部分彩票（ticket）时。

9.4 一个例子

为了让彩票调度（lottery scheduling）的动态行为更容易理解，我们现在简单研究一下两个互相竞争的作业的完成时间，这两个作业的彩票（ticket）数相同（都是 100），运行时间也相同（都是 R，我们会改变它的值）。

在这个场景中，我们希望两个作业大致同时完成；但由于彩票调度（lottery scheduling）的随机性，有时一个作业会比另一个先完成。为了量化这种差异，我们定义一个简单的**公平性指标**（fairness metric）F，它就是第一个作业的完成时间除以第二个作业的完成时间。例如，如果 $R = 10$，第一个作业在时刻 10 完成（第二个作业在时刻 20 完成），那么 $F = \frac{10}{20} = 0.5$。两个作业几乎同时完成时，F 会非常接近 1。在这个场景中，我们的目标就是：完全公平的调度程序应该做到 $F = 1$。

图 9.2（第 5 页）画出了两个作业的长度（R）从 1 变化到 1000 时，30 次试验的平均公平性（fairness），结果由本章末尾提供的模拟器生成。从图中可以看出，作业长度不太长

[2] 出人意料的是，正如 Björn Lindberg 指出的，要正确地做到这一点并不容易；详见 `http://stackoverflow.com/questions/2509679/how-to-generate-a-random-number-from-within-a-range`。

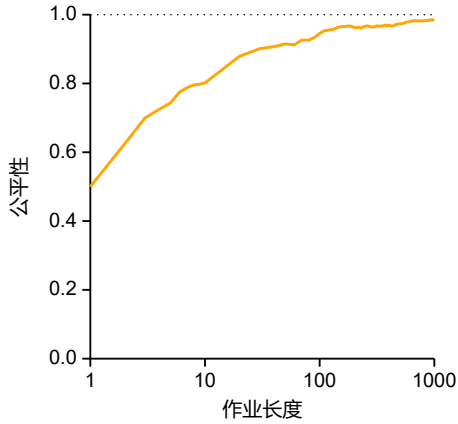


图 9.2：**彩票调度的公平性研究**

时，平均公平性可能相当低。只有当作业运行了相当多的时间片之后，彩票调度程序（lottery scheduler）才会接近期望的公平结果。

9.5 怎样分配彩票？

关于彩票调度（lottery scheduling），还有一个问题我们没有讨论：怎样给作业分配彩票（ticket）？这个问题很棘手，因为系统的行为显然在很大程度上取决于彩票（ticket）如何分配。一种办法是假定用户最清楚该怎么分；在这种情况下，每个用户分到一定数量的彩票（ticket），然后可以按自己的意愿把彩票分给自己运行的任何作业。然而，这个方案其实算不上方案：它根本没有告诉你该怎么做。因此，给定一组作业，“彩票（ticket）分配问题”仍然没有答案。

9.6 步长调度

你可能还会想：为什么要用随机性呢？正如前面看到的，随机性虽然让我们得到了一个简单（而且大致正确）的调度程序，但它偶尔无法给出完全准确的比例，在较短的时间尺度上尤其如此。为此，Waldspurger 发明了**步长调度**（stride scheduling），这是一种确定性的公平份额（fair-share）调度程序 [W95]。

步长调度（stride scheduling）也很简单。系统中的每个作业都有一个**步长**（stride），它和作业持有的彩票（ticket）数成反比。在前面的例子中，作业 A、B、C 分别持有 100、50 和 250 张彩票（ticket）；我们可以用某个大数分别除以每个进程分到的彩票（ticket）数，算出它们各自的步长（stride）。例如，用 10,000 分别除以这几个彩票（ticket）数，就得到 A、B、C 的步长（stride）：100、200 和 40。我们把这个值叫作每个进程的步长（stride）；每当一个进程运行一次，我们就把它的一个计数器（叫作它的 **pass** 值）加上它的步长（stride），以此记录它的总体进度。

然后，调度程序用步长（stride）和 pass 值来决定下一个该运行哪个进程。基本思路很简单：任何时候，都挑选目前 pass 值最小的进程来运行；运行完一个进程后，就把它的 pass 计数器加上它的步长（stride）。Waldspurger 给出了一个伪代码（pseudocode）实现 [W95]：

```
curr = remove_min(queue);    // 挑出 pass 值最小的客户
schedule(curr);              // 运行一个量子
curr->pass += curr->stride; // 用步长更新 pass 值
insert(queue, curr);         // 把 curr 放回队列
```

在我们的例子中，一开始有 A、B、C 三个进程，步长（stride）分别是 100、200 和 40，pass 值一开始都是 0。因此，最初任何一个进程都可以运行，因为它们的 pass 值同样最低。假设我们选了 A（随便选的；pass 值同样最低的进程都可以选）。A 运行；用完这个时间片后，我们把它的 pass 值更新为 100。接着运行 B，它的 pass 值随后变成 200。最后运行 C，它的 pass 值增加到 40。这时，算法会挑出 pass 值最低的进程，也就是 C，运行它，并把它的 pass 值更新为 80（你应该还记得，C 的步长（stride）是 40）。然后 C 会再运行一次（它的 pass 值仍然最低），pass 值升到 120。现在 A 会运行，pass 值更新为 200（和 B 的相等了）。接下来 C 又会运行两次，pass 值先后更新为 160 和 200。此时，所有的 pass 值又都相等了，这个过程会不断重复，永无止境。图 9.3（第 6 页）跟踪了调度程序随时间推移的行为。

从图中可以看出，C 运行了五次，A 运行了两次，B 只运行了一次，恰好和它们的彩票（ticket）数 250、100、50 成比例。彩票调度（lottery scheduling）随着时间推移，以概率的方式达到这些比例；步长调度（stride scheduling）则在每个调度周期结束时精确地达到这些比例。

所以你可能会想：既然步长调度（stride scheduling）这么精确，为什么还要用彩票调度（lottery scheduling）呢？原来，彩票调度（lottery scheduling）有一个步长调度（stride scheduling）所没有的好性质：没有全局状态。想象一下，在上面的步长调度（stride scheduling）例子进行到一半时，有一个新作业加入进来；它的 pass 值该设成多少？设成 0 吗？如果这样，它就会霸占 CPU。而用彩票调度（lottery scheduling）时，每个进程都没有全局状态；我们只需加入新进程以及它持有的彩票（ticket），更新那唯一一个记录彩票

Pass(A) （步长 = 100）	Pass(B) （步长 = 200）	Pass(C) （步长 = 40）	谁运行?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	…

图 9.3：**步长调度：跟踪过程**

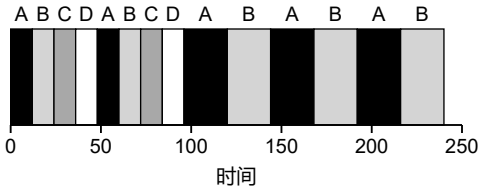


图 9.4：**CFS 的简单示例**

总数的全局变量，然后照常进行即可。这样一来，彩票调度（lottery scheduling）就能更容易、更合理地接纳新进程。

9.7 Linux 的完全公平调度程序（CFS）

尽管已经有了上面这些关于公平份额（fair-share）调度的早期工作，Linux 的一种（较早的）方法却用另一种方式实现了类似的目标。这个调度程序名叫**完全公平调度程序**（Completely Fair Scheduler，**CFS**）[J09]，它实现了公平份额（fair-share）调度，而且做得非常高效、可扩展。

为了达到高效的目标，CFS 力求在做调度决策上花尽可能少的时间，这既得益于它本身的设计，也得益于它巧妙地选用了非常适合这项任务的数据结构。最近的研究表明，调度程序的效率出人意料地重要；具体来说，Kanev 等人在一项针对 Google 数据中心的研究中指出，即使经过激进的优化，调度仍然用掉了整个数据中心大约 5% 的 CPU 时间 [K+15]。因此，尽可能降低这项开销，是现代调度程序架构的一个关键目标。

基本操作

大多数调度程序都基于固定时间片的概念，而 CFS 的做法有些不同。它的目标很简单：把一个 CPU 公平、均匀地分给所有相互竞争的进程。它通过一种简单的基于计数的技术做到这一点，这种技术叫作**虚拟运行时间**（virtual runtime，**`vruntime`**）。

每个进程运行时，都会累积 `vruntime`。在最基本的情况下，每个进程的 `vruntime` 以相同的速度增长，与物理（真实）时间成正比。做调度决策时，CFS 会挑选 `vruntime` 最低的进程作为下一个运行的进程。

这就引出了一个问题：调度程序怎么知道何时该停下当前运行的进程，转而运行下一个呢？这里的矛盾很明显：如果 CFS 切换得太频繁，公平性就会提高，因为即使在极短的时间窗口内，CFS 也能保证每个进程得到它那份 CPU，但代价是性能（上下文切换（context switch）太多）；如果 CFS 切换得不那么频繁，性能就会提高（上下文切换减少），但代价是短期内的公平性。

CFS 通过各种控制参数来处理这种矛盾。第一个是 **`sched_latency`**。CFS 用这个值来决定一个进程运行多久之后再考虑切换（实际上就是以动态的方式决定它的时间片）。`sched_latency` 的典型值是 48（毫秒）；CFS 用这个值除以 CPU 上正在运行的进程数（n），得到每个进程的时间片，从而确保在这段时间内，CFS 是完全公平的。

例如，如果有 $n = 4$ 个进程在运行，CFS 就用 `sched_latency` 除以 n，得到每个进程 12 ms 的时间片。然后 CFS 调度第一个作业并运行它，直到它用完 12 ms 的（虚拟）运行时间，再检查是否有 `vruntime` 更低的作业可以改为运行。在这个例子里确实有，于是 CFS 会切换到另外三个作业之一，依此类推。图 9.4（第 7 页）给出了一个例子：四个作业（A、B、C、D）按这种方式各运行了两个时间片；然后其中两个（C、D）完成，只剩下两个作业，它们随后以时间片轮转（Round-Robin）的方式各运行 24 ms。

但是，如果运行的进程“太多”了怎么办？那样不就会导致时间片太小，进而上下文切换太多吗？好问题！答案是肯定的。

为了解决这个问题，CFS 增加了另一个参数 **`min_granularity`**，它通常设成 6 ms 左右的值。CFS 绝不会把一个进程的时间片设得比这个值还小，以确保不会在调度开销上花费太多时间。

例如，如果有十个进程在运行，按最初的算法，要用 `sched_latency` 除以十来确定时间片，结果是 4.8 ms。但由于 `min_granularity` 的存在，CFS 会把每个进程的时间片改设为 6 ms。虽然在 48 ms 的目标调度延迟（`sched_latency`）内，CFS 不会（完全）做到完美公平，但会很接近，同时还能保持很高的 CPU 效率。

注意，CFS 使用周期性的时钟中断（timer interrupt），这意味着它只能在固定的时间间隔上做决策。这个中断（interrupt）触发得很频繁（比如每 1 ms 一次），让 CFS 有机会醒来，判断当前作业是否已经运行到头。如果某个作业的时间片不是时钟中断间隔的整数倍，也没关系；CFS 会精确地记录 `vruntime`，这意味着从长远来看，它终究会逼近理想的 CPU 分享。

权重（nice 值）

CFS 还允许控制进程的优先级（priority），让用户或管理员可以给某些进程分配更多的 CPU 份额。它不是靠彩票（ticket）做到这一点的，而是借助 UNIX 一种经典的机制：进程的 **nice** 级别。一个进程的 nice 参数可以设为 –20 到 +19 之间的任意值，默认是 0。nice 值为正表示优先级较低，为负表示优先级较高；唉，人要是太 nice（友善），就得不到那么多（调度上的）关注了。

CFS 把每个进程的 nice 值映射到一个权重，如下所示：

```
static const int prio_to_weight[40] = {
  /* -20 */ 88761, 71755, 56483, 46273, 36291,
  /* -15 */ 29154, 23254, 18705, 14949, 11916,
  /* -10 */  9548,  7620,  6100,  4904,  3906,
  /*  -5 */  3121,  2501,  1991,  1586,  1277,
  /*   0 */  1024,   820,   655,   526,   423,
  /*   5 */   335,   272,   215,   172,   137,
  /*  10 */   110,    87,    70,    56,    45,
  /*  15 */    36,    29,    23,    18,    15,
};
```

有了这些权重，我们就能（像之前一样）算出每个进程实际的时间片，不过这次考虑了它们之间的优先级差异。假设有 n 个进程，计算公式如下：

$$\text{time_slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched_latency} \tag{9.1}$$

我们来做个例子，看看这个公式是怎么起作用的。假设有两个作业 A 和 B。A 是我们最宝贵的作业，所以给它分配 –5 的 nice 值，让它的优先级更高；B 嘛，因为我们讨厌它[3]，所以只有默认的优先级（nice 值等于 0）。这意味着（查上表）weight_A 是 3121，而 weight_B 是 1024。如果你再计算每个作业的时间片，就会发现 A 的时间片大约是 `sched_latency` 的 $\frac{3}{4}$（因此是 36 ms），B 的大约是 $\frac{1}{4}$（因此是 12 ms）。

除了推广时间片的计算方法，CFS 计算 `vruntime` 的方式也要相应调整。下面是新的公式：它取进程 i 累计的实际运行时间（runtime_i），再按进程权重的反比进行缩放，也就是用默认权重 1024（weight_0）除以它的权重 weight_i。在我们这个一直在用的例子中，A 的 `vruntime` 累积速度是 B 的三分之一。

$$\text{vruntime}_i = \text{vruntime}_i + \frac{\text{weight}_0}{\text{weight}_i} \cdot \text{runtime}_i \tag{9.2}$$

上面这张权重表的构造有一个巧妙之处：只要 nice 值的差保持不变，这张表就能保持 CPU 的分配比例不变。例如，如果进程 A 的 nice 值改成 5（而不是 –5），进程 B 的 nice 值改成 10（而不是 0），CFS 调度它们的方式会和之前完全一样。自己算一算，看看为什么。

使用红黑树

如上所述，CFS 的一个重点是效率。对调度程序来说，效率有很多方面，其中一个就是这么简单：当调度程序需要找到下一个要运行的作业时，应该尽可能快地找到。列表这样简单的数据结构无法扩展：现代系统有时由上千个进程组成，因此每隔几毫秒就把一个长列表搜索一遍是一种浪费。

CFS 的解决办法是把进程放在一棵**红黑树**（red-black tree）中 [B72]。红黑树是众多平衡树中的一种；简单的二叉树在最坏的插入顺序下可能退化成列表那样的性能，与之相比，平衡树会多做一点额外的工作来保持较小的深度，从而保证操作的时间是对数级的（而不是线性的）。

CFS 并不把所有进程都放进这个结构，只有正在运行（或可运行）的进程才放在里面。如果一个进程进入睡眠（比如在等待一次 I/O 完成，或者等待一个网络包到达），它就会被移出这棵树，在别处另行记录。

来看一个例子，让这一点更清楚。假设有十个作业，它们的 `vruntime` 值分别是：1、5、9、10、14、18、17、21、22 和 24。如果把这些作业放在一个有序列表里，找出下一个要运行的作业很简单：直接取出第一个元素就行。但是，把那个作业（按顺序）放回列表时，我们就得扫描列表，找到正确的插入位置，这是一个 $O(n)$ 的操作。任何查找的效率也都很低，平均同样需要线性时间。

[3]是的，是的，我们这里是故意用错语法的（原文“because we hates it”），请不要来提交错误修正。为什么这么写？嗯，只不过是极其轻微地致敬一下《指环王》以及我们最喜欢的反英雄咕噜，没什么好激动的。

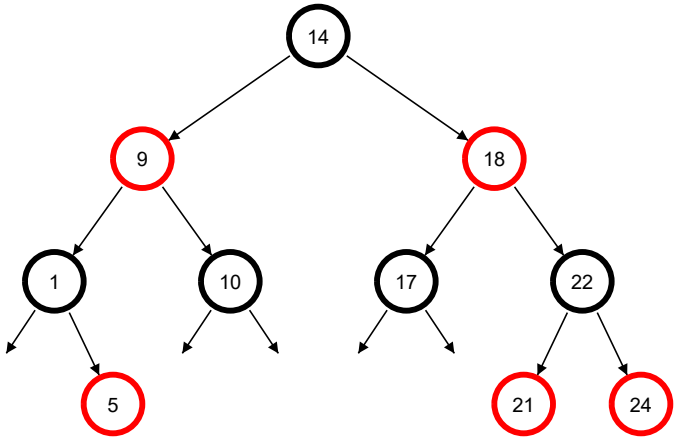


图 9.5：**CFS 红黑树**

把同样的值放进红黑树，大多数操作都会更高效，如图 9.5（第 10 页）所示。进程在树中按 `vruntime` 排序，大多数操作（比如插入和删除）的时间都是对数级的，即 $O(\log n)$。当 n 达到上千时，对数级明显比线性的效率高。

处理 I/O 和睡眠的进程

总是挑选 `vruntime` 最低的进程来运行，会给那些睡眠了很长时间的作业带来一个问题。想象有两个进程 A 和 B，其中一个（A）一直在运行，另一个（B）睡眠了很长时间（比如 10 秒）。当 B 醒来时，它的 `vruntime` 会比 A 的落后 10 秒；因此（如果不小心的话），B 会在接下来的 10 秒里霸占 CPU 来追赶 A，实际上让 A 陷入了饥饿（starvation）。

CFS 处理这种情况的办法是，在作业醒来时修改它的 `vruntime`。具体来说，CFS 把这个作业的 `vruntime` 设为树中能找到的最小值（别忘了，树里只有正在运行的作业）[B+18]。CFS 由此避免了饥饿，但也付出了代价：频繁地短暂睡眠的作业永远得不到它们应得的公平份额（fair share） [AC97]。

CFS 的其他有趣之处

CFS 还有许多其他特性，多到在本书的这个位置无法一一讨论。它有大量用来提高缓存（cache）性能的启发式方法，有高效处理多个 CPU 的策略（本书后面会讨论），还能以大组进程为单位进行调度（而不是把每个进程当作一个独立的实体），此外还有许多其他有趣的特性。想了解更多，请阅读最近的研究，可以从 Bouron 的论文 [B+18] 读起。

9.8 小结

我们介绍了比例份额（proportional-share）调度的概念，并简要讨论了三种方法：彩票调度（lottery scheduling）、步长调度（stride scheduling）和 CFS。彩票调度（lottery

提示：在合适的时候使用高效的数据结构

很多时候，用列表就够了。很多时候，又不够。知道什么时候该用哪种数据结构，是优秀工程能力的一个标志。在这里讨论的例子中，早期调度程序中使用的简单列表在现代系统上效果很差，在数据中心里负载很重的服务器上尤其如此。这类系统有成千上万个活跃进程；每隔几毫秒就在每个核心上搜索一遍长列表，来找出下一个要运行的作业，会浪费宝贵的 CPU 周期。我们需要一种更好的结构，而 CFS 加入了一个出色的红黑树实现，提供了这样一种结构。更一般地说，在为你正在构建的系统选择数据结构时，要仔细考虑它的访问模式和使用频率；理解了这些，你就能为手头的任务实现合适的结构。

scheduling）巧妙地利用随机性来实现比例份额（proportional-share）；步长调度（stride scheduling）则以确定性的方式做到这一点。CFS 是本章讨论的唯一一个“真正的”调度程序，它有点像带动态时间片的加权时间片轮转（Round-Robin），但为可扩展性（scalability）和高负载下的良好表现而构建；多年来，它一直是使用最广泛的公平份额（fair-share）调度程序。

没有哪个调度程序是万能的，公平份额（fair-share）调度程序自然也分到了它“公平的一份”问题。一个问题是，这些方法和 I/O 配合得不太好 [AC97]；如上所述，偶尔执行 I/O 的作业可能得不到它应得的那份 CPU。另一个问题是，它们留下了彩票（ticket）或优先级分配这个难题没有解决：你怎么知道该给浏览器分配多少张彩票，或者该把文本编辑器的 nice 值设成多少？其他通用的调度程序会自动处理这些问题，因此可能更容易部署。

好消息是，在许多领域里，这些问题并不是主要矛盾，比例份额（proportional-share）调度程序在那里能发挥很大的作用。例如，在**虚拟化**（virtualized）的数据中心（或**云**，cloud）中，你可能想把四分之一的 CPU 周期分给 Windows 虚拟机（virtual machine），其余的分给你的 Linux 主系统，这时比例分享就既简单又有效。这个思想也可以推广到其他资源上；详见 Waldspurger 的论文 [W02]。

最后，Linux 的世界总在变化。从 Linux 6.6 起，默认调度程序换成了 Earliest Eligible Virtual Deadline First（EEVDF）调度程序，这是另一种公平份额（fair-share）方法。想了解更多，就去读读关于 EEVDF 的资料吧 [Z23, SA95]。你还以为学完了呢！

 © 2008–26, ARPACI-DUSSEAU

参考文献

[AC97] "Extending Proportional-Share Scheduling to a Network of Workstations" by Andrea C. Arpaci-Dusseau and David E. Culler. PDPTA'97, June 1997. *A paper by one of the authors on how to extend proportional-share scheduling to work better in a clustered environment.*

[B+18] "The Battle of the Schedulers: FreeBSD ULE vs. Linux CFS" by J. Bouron, S. Chevalley, B. Lepers, W. Zwaenepoel, R. Gouicem, J. Lawall, G. Muller, J. Sopena. USENIX ATC '18, July 2018, Boston, Massachusetts. *A recent, detailed work comparing Linux CFS and the FreeBSD schedulers. An excellent overview of each scheduler is also provided. The result of the comparison: inconclusive. In some cases CFS was better, and in others, ULE (the BSD scheduler), was. Sometimes in life there are no easy answers.*

[B72] "Symmetric binary B-Trees: Data Structure And Maintenance Algorithms" by Rudolf Bayer. Acta Informatica, Volume 1, Number 4, December 1972. *A cool balanced tree introduced before you were born (most likely). One of many balanced trees out there; study your algorithms book for more alternatives!*

[D82] "Why Numbering Should Start At Zero" by Edsger Dijkstra, August 1982. Available: http://www.cs.utexas.edu/users/EWD/ewd08xx/EWD831.PDF. *A short note from E. Dijkstra, one of the pioneers of computer science. We'll be hearing much more on this guy in the section on Concurrency. In the meanwhile, enjoy this note, which includes this motivating quote: "One of my colleagues — not a computing scientist — accused a number of younger computing scientists of 'pedantry' because they started numbering at zero." The note explains why doing so is logical.*

[K+15] "Profiling A Warehouse-scale Computer" by S. Kanev, P. Ranganathan, J. P. Darago, K. Hazelwood, T. Moseley, G. Wei, D. Brooks. ISCA '15, June, 2015, Portland, Oregon. *A fascinating study of where the cycles go in modern data centers, which are increasingly where most of computing happens. Almost 20% of CPU time is spent in the operating system, 5% in the scheduler alone!*

[J09] "Inside The Linux 2.6 Completely Fair Scheduler" by M. Tim Jones. December 15, 2009. https://pages.cs.wisc.edu/%7Eremzi/OSTEP/Citations/inside-cfs.pdf. *This is a simple overview of CFS from its earlier days. CFS was created by Ingo Molnar in a short burst of creativity which led to a 100K kernel patch developed in 62 hours.*

[KL88] "A Fair Share Scheduler" by J. Kay and P. Lauder. CACM, Volume 31 Issue 1, January 1988. *An early reference to a fair-share scheduler.*

[SA95] "Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation" by Ion Stoica and Hussein M. Abdel-Wahab. Technical report, CS Department, Old Dominion University, TR-95-22. *The technical report that introduced EEVDF. Ion Stoica has since gone on to having great impact in systems, including with Apache Spark, the founding of Databricks, and much more.*

[WW94] "Lottery Scheduling: Flexible Proportional-Share Resource Management" by Carl A. Waldspurger and William E. Weihl. OSDI '94, November 1994. *The landmark paper on lottery scheduling that got the systems community re-energized about scheduling, fair sharing, and the power of simple randomized algorithms.*

[W95] "Lottery and Stride Scheduling: Flexible Proportional-Share Resource Management" by Carl A. Waldspurger. Ph.D. Thesis, MIT, 1995. *The award-winning thesis of Waldspurger's that outlines lottery and stride scheduling. If you're thinking of writing a Ph.D. dissertation at some point, you should always have a good example around, to give you something to strive for: this is such a good one.*

[W02] "Memory Resource Management in VMware ESX Server" by Carl A. Waldspurger. OSDI '02, Boston, Massachusetts. *The paper to read about memory management in VMMs (a.k.a., hypervisors). In addition to being relatively easy to read, the paper contains numerous cool ideas about this new type of VMM-level memory management.*

[Z23] "The EEVDF Scheduler" by Peter Zijlstra. Please go to this URL for details on EEVDF: https://docs.kernel.org/scheduler/sched-eevdf.html. *Linux is always a'changing!*

 © 2008–26, ARPACI-DUSSEAU

作业（模拟）

程序 `lottery.py` 可以让你观察彩票调度程序（lottery scheduler）是如何工作的。详情请见 README。

问题

1. 计算 3 个作业、随机种子分别为 1、2、3 时的模拟结果。
2. 现在用两个特定的作业运行：两个作业的长度都是 10，但一个（作业 0）只有 1 张彩票（ticket），另一个（作业 1）有 100 张（例如 `-l 10:1,10:100`）。彩票（ticket）数这么不均衡时，会发生什么？作业 0 会在作业 1 完成之前运行吗？多久运行一次？总的来说，这样的彩票（ticket）不均衡会对彩票调度（lottery scheduling）的行为产生什么影响？
3. 运行两个作业，长度都是 100，彩票（ticket）数也都是 100（`-l 100:100,100:100`）。这时调度程序有多不公平？用一些不同的随机种子运行，求出（概率意义上的）答案；用一个作业比另一个作业早完成多少来衡量不公平程度。
4. 随着量子大小（quantum size，`-q`）变大，你对上一题的答案会如何变化？
5. 你能画出本章中那张图的一个版本吗？还有什么值得探索的？换成步长调度程序（stride scheduler），这张图会是什么样子？

 © 2008–26, ARPACI-DUSSEAU